

Review: "Regexes are Hard: Decision-making, Difficulties, and Risks in Programming Regular Expressions"

Michael IV et al. (2023) · arXiv:2303.02555

1. Introduction

This paper, authored by Louis G. Michael IV, James Donohue, James C. Davis, Dongyoon Lee, and Francisco Servant, was presented at the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE) in 2019 and later published on arXiv in 2023. The central topic is how professional software developers actually work with regular expressions (regexes) – a widely-used tool that, according to the authors, appears in more than a third of Python and JavaScript projects. While prior research mostly focused on the readability of regexes as code artifacts, this study shifts the lens toward the humans using them: what decisions they make, what trips them up, and what risks they are often unaware of.

2. Methodology

The authors used a mixed-methods approach. First, they surveyed 279 professional developers from companies ranging from startups to Fortune 500 firms. The surveys contained both multiple-choice questions (for quantitative perspective) and open-ended questions (analyzed qualitatively using open coding, a technique from grounded theory). To go deeper, they conducted semi-structured interviews with 17 developers who had previously completed the surveys. The interview guides were not rigid scripts but rather flexible topic maps, allowing for genuine discovery. Responses were independently coded by multiple researchers and reconciled to reduce interpretation bias.

3. Results

The findings are organized around five research questions. Developers widely view regexes as valuable – 88% agreed they are important for their work – yet 65% found them daunting when first learning. Most developers (70%) disagreed that regexes are more readable than other code. When composing regexes, the choice between writing from scratch and reusing an existing one depends mainly on problem commonality and perceived reliability of the source. Validation is often shallow, particularly for internal tools. The most alarming result: only 38% of surveyed developers had heard of ReDoS (Regular Expression Denial of Service), a performance-based security vulnerability that can be exploited through malicious input. Even those who knew about it did not consistently apply effective countermeasures.

4. Key Insights

Insight 1 – The gap between perceived and actual portability. Many developers assume regexes work the same across programming languages. In reality, different regex dialects (Python `re`, JavaScript, Go, etc.) have divergent syntax and semantics, which can cause unexpected behavior when a regex is reused across environments. This matters directly for anyone building multi-language systems or working on projects that share validation logic. Knowing this changes how one

should approach regex reuse: always test in the target environment, not just where it was written.

Insight 2 – ReDoS is largely unknown, even among senior developers. The fact that under 40% of professionals are aware of ReDoS is striking. It means this vulnerability is likely present in a lot of production code without anyone noticing, because it only manifests under adversarial input. For anyone working on ML or data systems that parse user-provided strings, this is a concrete risk worth understanding – input sanitization and avoiding super-linear regex patterns are both simple and effective mitigations.

Insight 3 – Developers validate regexes poorly by default. Most developers test regexes against examples they already have, which means they only cover the input space they can imagine. Edge cases, by definition, are easy to miss this way. This is relevant not only for regex validation but as a broader pattern: the way people build confidence in their code is often based on available data, not on systematic boundary analysis. Being aware of this tendency helps build better testing habits, whether it is regex-specific or general.

5. Conclusion

This paper makes a meaningful contribution by shifting focus from regexes as technical artifacts to regexes as something humans struggle with. The findings are grounded in real developer experience across a wide range of companies and experience levels, which makes them credible and actionable. The authors also lay out a clear research agenda: semantic regex search engines, automated documentation, regex metrics, and better tooling for detecting performance vulnerabilities. For practitioners, the main takeaway is simple: regexes deserve the same level of care as any other production code – proper testing, documentation, and a basic awareness of security risks – because without that, the problems described in this paper are almost guaranteed to appear!